
UADE

UADE

Exposición de Experto

Temas a desarrollar

- Taxonomía de los *Code Smells*
- Técnicas de refactorización
- Ejemplos de *Bad Smells* en código
- Introducción a GRASP
- Síntesis

UADE

Taxonomía de los *Code Smells*

Taxonomía de los *Code Smells*

Según la taxonomía de Mäntylä, los malos olores en el código se dividen en cinco categorías principales:	
1. Inflados (*Bloaters*)	• Código innecesariamente grande o complejo.
2. Acopladores (*Couplers*)	• Clases y métodos que dependen demasiado unos de otros.
3. Prescindibles (*Dispensables*)	• Código innecesario o redundante.
4. Frustradores del Cambio (*Change Preventers*)	• Código que dificulta la modificación y el mantenimiento.
5. Abuso de la POO (*Object-Orientation Abusers*)	• Mal uso de los principios de la Programación Orientada a Objetos (POO).

UADE

UADE

Técnicas de refactorización

Técnicas de refactorización

Para corregir los malos olores, se sugieren varias técnicas de refactorización:

1. Composición de métodos	• Extraer métodos para hacer el código más claro y reutilizable.
2. Movimiento de funcionalidad entre objetos	• Reubicar métodos y datos donde tengan más sentido.
3. Organización de datos	• Convertir arreglos en objetos o eliminar números mágicos.
4. Simplificación de expresiones condicionales	• Reemplazar condicionales anidados con cláusulas de guardia.
5. Mejora de llamadas a métodos	• Introducir objetos de parámetros, ocultar métodos innecesarios.
6. Manejo de generalización	• Extraer clases, eliminar intermediarios innecesarios.



UADE

Ejemplos de *Bad Smells* en Código

Bloaters (Inflados)

Long Method:	Métodos demasiado largos y difíciles de entender.
Large Class:	Clases con demasiadas responsabilidades.
Primitive Obsession:	Uso excesivo de tipos de datos primitivos en lugar de objetos.
Long Parameter List:	Métodos con demasiados parámetros, lo que los hace difíciles de entender y usar.
Data Clumps:	Grupos de variables que deberían encapsularse en un objeto.



Object-Orientation Abusers (Abuso de la POO)

Switch Statements:	• Uso excesivo de condicionales en lugar de polimorfismo.
Temporary Field:	• Variables temporales usadas solo en algunas ocasiones.
Refused Bequest:	• Clases que heredan de otras, pero no usan sus métodos.
Alternative Classes with Different Interfaces:	• Clases con funcionalidades similares, pero interfaces diferentes.

UADE

Change Preventers (Frustradores del Cambio)

Divergent Change:	• Una clase que cambia por múltiples razones, violando el principio de responsabilidad única (SRP).
Shotgun Surgery:	• Un pequeño cambio que obliga a modificar muchas partes del código.
Parallel Inheritance Hierarchies:	• Jerarquías de herencia paralelas que generan complejidad innecesaria.

UADE



Dispensables (Prescindibles)

Comments:	Código que requiere demasiados comentarios para ser entendido.
Duplicate Code:	Código duplicado en diferentes partes del sistema.
Lazy Class:	Clases que tienen poco o ningún comportamiento útil.
Data Class:	Clases que solo contienen datos sin lógica.
Dead Code:	Código que no se usa y debería eliminarse.
Speculative Generality:	Código escrito para funcionalidades futuras que quizás nunca se necesiten.

UADE

Couplers (Acopladores)

Feature Envy:	• Un método que usa más los atributos de otra clase que los de su propia clase.
Inappropriate Intimacy:	• Clases que acceden a detalles internos de otras clases.
Message Chain:	• Llamadas encadenadas que crean dependencias difíciles de manejar.
Middle Man:	• Clases que solo sirven como intermediarios sin agregar valor.



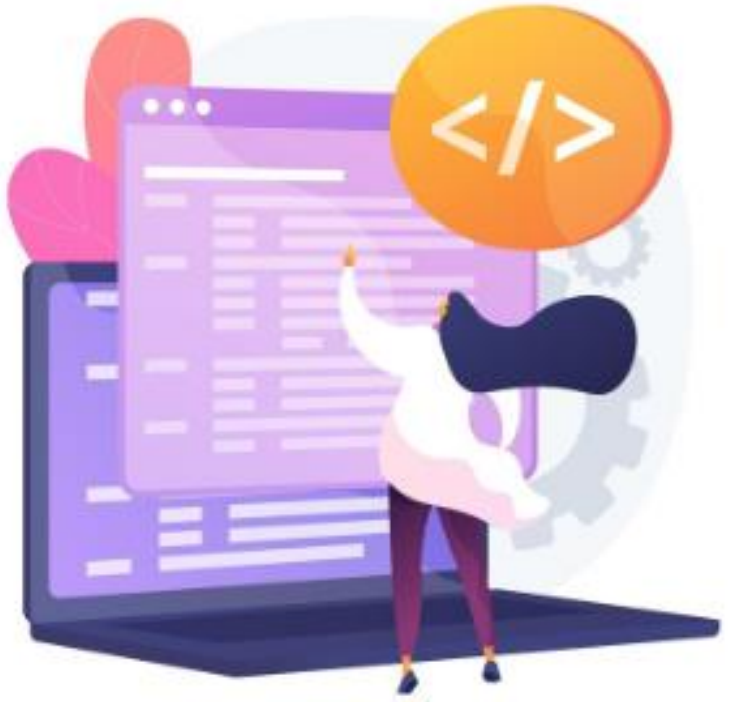
UADE

Introducción a GRASP – Patrones de Responsabilidad

¿Qué es GRASP?

Son nueve patrones que nos ayudan a decidir quién debería hacer qué en un diseño orientado a objetos.
Se enfocan en la asignación de responsabilidades para lograr código flexible, claro y mantenible.

Ahora veamos cuales son:



Patrones principales de GRASP:

***Expert* (Experto)**

→ Asignar la responsabilidad a quien tiene la información necesaria.

***Creator* (Creador)**

→ Una clase crea otra si la usa o tiene su información.

***Controller* (Controlador)**

→ Centraliza la coordinación entre el sistema y los objetos.

***Polymorphism* (Polimorfismo)**

→ Permite que distintas clases respondan de forma diferente al mismo mensaje.

Patrones principales de GRASP:

Low Coupling **(Bajo acoplamiento)**

→ Minimizar dependencias entre clases.

High Cohesion **(Alta cohesión)**

→ Que cada clase tenga una única responsabilidad bien definida.

Pure Fabrication **(Fabricación Pura)**

→ Crear clases artificiales para mejorar diseño sin representar entidades del dominio.

Indirection **(Indirección)**

→ Intermediarios que desacoplan objetos.

Protected Variations **(Variaciones Protegidas)**

→ Encapsular lo que puede cambiar para protegerse del impacto.



UADE

Síntesis

Síntesis

- **Code Smells**
  - Indicadores de problemas en el código que, si bien no siempre son errores, afectan su mantenibilidad y calidad.
  - **Ejemplos comunes: c**ódigo duplicado, métodos demasiado largos, clases con demasiadas responsabilidades (*God Class*), dependencias excesivas entre módulos. **Objetivo:** detectarlos y refactorizar para mejorar claridad, flexibilidad y evitar errores futuros.
- **GRASP (*General Responsibility Assignment Software Patterns*)**
  - Conjunto de principios para asignar responsabilidades en diseño orientado a objetos.
  - **Principios clave:**
    - ***Creator*:** quién debe crear un objeto.
    - ***Information Expert*:** asignar responsabilidades al que tiene la información.
    - ***Controller*:** manejo de la interacción entre UI y lógica.
    - ***Low Coupling / High Cohesion*:** bajo acoplamiento y alta cohesión.
    - ***Polymorphism*:** usar polimorfismo para manejar variaciones.
    - ***Pure Fabrication*:** clases auxiliares para mejorar diseño.
    - ***Indirection*:** introducir intermediarios para reducir dependencias.
    - ***Protected Variations*:** proteger contra cambios inesperados.

- **Code Smells:** detectar síntomas de mal diseño.
- **GRASP:** aplicar principios para prevenirlos o corregirlos.

Bibliografía utilizada para este recurso



LARMAN, Craig; GARCÍA MOLINA, Jesús. ***UML y patrones: una introducción al análisis y diseño orientado a objetos y al proceso unificado.*** **2a ed. 2003. Madrid: Pearson Educación. xxv, 590 p. ISBN: 9788420534381**

UADE

¡Muchas gracias!